

Red Black Tree:

Code:

```
// CH.SC.U4CSE24135
// Implementation of Red-Black Tree with insertion and inorder traversal

#include <stdio.h>
#include <stdlib.h>

#define RED 1
#define BLACK 0

struct Node {
    int data;
    int color;
    struct Node *left, *right, *parent;
};

struct Node* createNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->color = RED;
    node->left = node->right = node->parent = NULL;
    return node;
}

void leftRotate(struct Node **root, struct Node *x) {
    struct Node *y = x->right;
    x->right = y->left;

    if (y->left != NULL)
        y->left->parent = x;

    y->parent = x->parent;
```

```

    if (x->parent == NULL)
        *root = y;
    else if (x == x->parent->left)
        x->parent->left = y;
    else
        x->parent->right = y;

    y->left = x;
    x->parent = y;
}

void rightRotate(struct Node **root, struct Node *y) {
    struct Node *x = y->left;
    y->left = x->right;

    if (x->right != NULL)
        x->right->parent = y;

    x->parent = y->parent;

    if (y->parent == NULL)
        *root = x;
    else if (y == y->parent->left)
        y->parent->left = x;
    else
        y->parent->right = x;

    x->right = y;
    y->parent = x;
}

```

```

void fixInsert(struct Node **root, struct Node *z) {
    while (z->parent != NULL && z->parent->color == RED) {
        if (z->parent == z->parent->parent->left) {
            struct Node *y = z->parent->parent->right;

            if (y != NULL && y->color == RED) {
                z->parent->color = BLACK;
                y->color = BLACK;
                z->parent->parent->color = RED;
                z = z->parent->parent;
            } else {
                if (z == z->parent->right) {
                    z = z->parent;
                    leftRotate(root, z);
                }
                z->parent->color = BLACK;
                z->parent->parent->color = RED;
                rightRotate(root, z->parent->parent);
            }
        } else {
            struct Node *y = z->parent->parent->left;

            if (y != NULL && y->color == RED) {
                z->parent->color = BLACK;
                y->color = BLACK;
                z->parent->parent->color = RED;
                z = z->parent->parent;
            } else {
                if (z == z->parent->left) {
                    z = z->parent;
                    rightRotate(root, z);
                }
                z->parent->color = BLACK;
                z->parent->parent->color = RED;
                leftRotate(root, z->parent->parent);
            }
        }
    }
}

```

```

    }
}
(*root)->color = BLACK;
}

void insert(struct Node **root, int data) {
    struct Node *z = createNode(data);
    struct Node *y = NULL;
    struct Node *x = *root;

    while (x != NULL) {
        y = x;
        if (z->data < x->data)
            x = x->left;
        else
            x = x->right;
    }

    z->parent = y;

    if (y == NULL)
        *root = z;
    else if (z->data < y->data)
        y->left = z;
    else
        y->right = z;

    fixInsert(root, z);
}

```

```

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    printf("CH.SC.U4CSE24135\n");

    struct Node *root = NULL;

    insert(&root, 157);
    insert(&root, 110);
    insert(&root, 147);
    insert(&root, 122);
    insert(&root, 111);
    insert(&root, 149);
    insert(&root, 151);
    insert(&root, 141);
    insert(&root, 112);
    insert(&root, 117);
    insert(&root, 133);

    printf("Inorder Traversal:\n");
    inorder(root);

    return 0;
}

```

Output:

```

[keermanipamisetty@Keermanis-MacBook-Air DAA LAB % gcc red_black_tree.c -o red_black_tree
[keermanipamisetty@Keermanis-MacBook-Air DAA LAB % ./red_black_tree
CH.SC.U4CSE24135
Inorder Traversal:
110 111 112 117 122 133 141 147 149 151 157 %

```

AVL Tree

Code:

```
// CH.SC.U4CSE24135
// Implementation of AVL Tree insertion and inorder traversal

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    int height;
    struct Node *left, *right;
};

int max(int a, int b) {
    return (a > b) ? a : b;
}

int height(struct Node *n) {
    return (n == NULL) ? 0 : n->height;
}

struct Node* createNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = node->right = NULL;
    node->height = 1;
    return node;
}

struct Node* rightRotate(struct Node *y) {
    struct Node *x = y->left;
    struct Node *T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
```

```

    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

struct Node* leftRotate(struct Node *x) {
    struct Node *y = x->right;
    struct Node *T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

int getBalance(struct Node *n) {
    return (n == NULL) ? 0 : height(n->left) - height(n->right);
}

struct Node* insert(struct Node* node, int data) {
    if (node == NULL)
        return createNode(data);

    if (data < node->data)
        node->left = insert(node->left, data);
    else if (data > node->data)
        node->right = insert(node->right, data);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));
}

```

```

    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

struct Node* leftRotate(struct Node *x) {
    struct Node *y = x->right;
    struct Node *T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

int getBalance(struct Node *n) {
    return (n == NULL) ? 0 : height(n->left) - height(n->right);
}

struct Node* insert(struct Node* node, int data) {
    if (node == NULL)
        return createNode(data);

    if (data < node->data)
        node->left = insert(node->left, data);
    else if (data > node->data)
        node->right = insert(node->right, data);
    else
        return node;

    node->height = 1 + max(height(node->left), height(node->right));
}

```



```

    int balance = getBalance(node);

    if (balance > 1 && data < node->left->data)
        return rightRotate(node);

    if (balance < -1 && data > node->right->data)
        return leftRotate(node);

    if (balance > 1 && data > node->left->data) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    if (balance < -1 && data < node->right->data) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

```

```

int main() {
    printf("CH.SC.U4CSE24135\n");

    struct Node *root = NULL;

    root = insert(root, 157);
    root = insert(root, 110);
    root = insert(root, 147);
    root = insert(root, 122);
    root = insert(root, 111);
    root = insert(root, 149);
    root = insert(root, 151);
    root = insert(root, 141);
    root = insert(root, 112);
    root = insert(root, 117);
    root = insert(root, 133);

    printf("Inorder Traversal:\n");
    inorder(root);

    return 0;
}

```

Output:

```

keermanipamisetty@Keermanis-MacBook-Air DAA LAB % gcc avl_tree.c -o avl_tree
[keermanipamisetty@Keermanis-MacBook-Air DAA LAB % ./avl_tree
CH.SC.U4CSE24135
Inorder Traversal:
110 111 112 117 122 133 141 147 149 151 157 %

```